

INSTITUTO FEDERAL DO PARANÁ

Campus Pinhais

Ministério da Educação

CURSO BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Projeto de Extensão II

Rodrigo Machado

RELATÓRIO DE ATIVIDADES

Evolução do SoundCon: autenticação em nuvem, estação de áudio multitrack na web e arquitetura MVC

PINHAIS

2026

Rodrigo Machado

RELATÓRIO DE ATIVIDADES

Evolução do SoundCon: autenticação em nuvem, estação de áudio multitrack na web e arquitetura MVC

Relatório de atividades extensionistas, apresentado à disciplina de Projeto de Extensão II, do curso de Bacharelado em Ciência da Computação do Instituto Federal do Paraná — Campus Pinhais, sob a orientação dos professores Guilherme Werneck e João Orlando.

PINHAIS

2026

SUMÁRIO

1 INTRODUÇÃO	4
1.1 Objetivo	4
2 CONTEXTUALIZAÇÃO	5
3 DESCRIÇÕES DAS ATIVIDADES REALIZADAS	6
3.1 Materiais	6
3.2 Método	6
4 RESULTADOS E DISCUSSÕES	8
5 CONSIDERAÇÕES FINAIS	9
6 REFERÊNCIAS	10

1 INTRODUÇÃO

O **SoundCon** é uma aplicação web de processamento de áudio que, em sua origem, oferecia conversão entre formatos e ajuste de tonalidade (pitch shift) por meio de um backend em Node.js/Express com FFmpeg e um frontend em React. Este relatório apresenta o conjunto de atividades realizadas na etapa final do projeto, no âmbito da disciplina de Projeto de Extensão II, em que a aplicação evoluiu de uma ferramenta de conversão pontual para uma **plataforma de áudio mais completa, segura e escalável**.

O trabalho concentrou-se em cinco frentes complementares: (i) a reestruturação do backend para a arquitetura **MVC**, visando manutenção e escalabilidade futuras; (ii) a substituição da autenticação frágil por **autenticação gerenciada na nuvem (Supabase Auth)** com banco de dados **PostgreSQL** no lugar do SQLite; (iii) a construção de uma **estação de áudio multitrack na web (DAW)** ao estilo de softwares profissionais, com preview de efeitos em tempo real e renderização final pelo FFmpeg; (iv) o aprimoramento da **qualidade de conversão** com controles por codec; e (v) uma **avaliação de UX/UI** com correções priorizadas de usabilidade e acessibilidade.

A justificativa do trabalho está no caráter extensionista e formativo do projeto: entregar um serviço web útil e acessível, que funcione mesmo em computadores modestos, ao mesmo tempo em que se aplicam, na prática, conceitos de engenharia de software, processamento digital de sinais, segurança de aplicações e otimização baseada em estruturas de dados.

1.1 Objetivo

Evoluir o SoundCon para uma plataforma de áudio web robusta e escalável, contemplando:

- reorganização do backend em arquitetura MVC, sem quebra de compatibilidade da API;
- autenticação real com sessão (JWT) gerenciada pelo Supabase e migração do banco para PostgreSQL com regras de acesso (RLS);
- uma estação de mixagem multitrack no navegador, otimizada para hardware modesto, com cadeia de efeitos de áudio;
- conversão de áudio de maior qualidade, com parâmetros configuráveis por formato;
- avaliação de experiência do usuário (UX/UI) e implementação das correções de maior impacto;

- manutenção da estabilidade por meio de testes automatizados e validações de build.

2 CONTEXTUALIZAÇÃO

A evolução proposta apoia-se em quatro áreas de conhecimento, descritas a seguir.

Autenticação e segurança de aplicações web. Sistemas modernos separam a identidade do usuário da lógica de negócio por meio de provedores de autenticação que emitem tokens **JWT (JSON Web Token)**. A plataforma **Supabase** oferece autenticação gerenciada sobre um banco **PostgreSQL**, com **RLS (Row Level Security)** — política que restringe o acesso às linhas de uma tabela ao próprio dono do dado. Esse modelo dispensa o armazenamento manual de senhas (com `bcrypt`) e reduz a superfície de erro, mantendo segredos sensíveis (a *service-role key*) apenas no servidor, enquanto o cliente utiliza somente a chave pública (*anon key*).

Processamento digital de áudio. Aplicar equalização e efeitos de dinâmica a um sinal pode ser feito de duas maneiras complementares: em **tempo real no navegador**, com a **Web Audio API** (nós como *BiquadFilterNode*, *DynamicsCompressorNode*, *GainNode*, *StereoPannerNode* e *AnalyserNode*), permitindo ao usuário ouvir e visualizar as mudanças instantaneamente; e em **renderização final** com o **FFmpeg**, que aplica os filtros (*equalizer*, *superequalizer*, *highpass*, *lowpass*, *acompressor*, *loudnorm*, entre outros) com fidelidade ao arquivo exportado. Alguns efeitos têm equivalente exato em ambos os mundos; outros, sem contraparte nativa no navegador, são aplicados apenas no render.

Arquitetura de software. O padrão **MVC (Model–View–Controller)** e a separação em camadas (rotas, controladores, serviços, modelos, middlewares, utilitários) favorecem a manutenção e a escalabilidade, isolando responsabilidades: controladores finos cuidam da entrada/saída HTTP, enquanto a regra de negócio concentra-se nos serviços e o acesso a dados nos modelos.

Otimização e estruturas de dados. Para que uma estação de áudio funcione de forma fluida em máquinas comuns, é necessário evitar travamentos e coletas de lixo (GC) no caminho crítico do áudio. Empregam-se estruturas e técnicas como **buffer circular (ring buffer)** sobre *typed arrays* para o histórico dos medidores, **pilha** para desfazer/refazer, **Web Workers** para cálculo fora da thread principal, **transferable ArrayBuffers** (cópia zero) e renderização em **canvas** desacoplada da taxa de áudio por um único laço de animação com limitação adaptativa de quadros.

3 DESCRIÇÕES DAS ATIVIDADES REALIZADAS

3.1 Materiais

Backend: Node.js, Express 5, Multer (upload), FFmpeg embutido via *@ffmpeg-installer/ffmpeg*, *@supabase/supabase-js* (Auth + PostgreSQL), *dotenv* (variáveis de ambiente) e o executor de testes nativo *node --test*.

Frontend: React 19, Vite (distribuição *rolldown-vite*), React Router 7, *@supabase/supabase-js*, **Web Audio API**, **Web Workers**, elemento **Canvas** e ESLint para análise estática.

Banco de dados: PostgreSQL gerenciado pelo Supabase, com RLS e *trigger* de criação automática de perfil no cadastro.

Documentação de referência: documentação oficial do FFmpeg (filtros e *resampler*), documentação do Supabase, *MDN Web Docs* (Web Audio API) e as normas de apresentação de trabalhos acadêmicos do IFPR.

Ferramenta de apoio: script *dev.js* (Node puro, sem dependências) que sobe o backend e o frontend simultaneamente com saída identificada por serviço.

3.2 Método

As atividades foram conduzidas em ciclos de análise, implementação e validação, organizados em uma fundação técnica seguida de quatro fases priorizadas. Cada etapa foi verificada por testes e/ou *build* antes de avançar.

Fundação — Reestruturação do backend para MVC. O *server.js*, que misturava rotas, regras de negócio e acesso a dados, foi reduzido a um *bootstrap*. Toda a aplicação passou para *backend/src/*, organizada em *config/* (ambiente, caminhos, FFmpeg, cliente Supabase), *routes/*, *controllers/* (finos), *services/* (regra de negócio — processamento de áudio, mixagem, logs), *models/* (acesso a dados), *middlewares/* (autenticação JWT, upload, tratamento central de erros) e *utils/* (execução do FFmpeg com *timeout*, remoção segura de arquivos). Os utilitários existentes foram **movidos e reaproveitados**, não reescritos, e a migração ocorreu **sem alterar os contratos da API**, mantendo os testes legados aprovados. No frontend, criou-se um **cliente de API centralizado** (*services/api.js*) com a base configurável por variável de ambiente, eliminando URLs fixas espalhadas pelas telas.

Fase 1 — Autenticação com Supabase e migração para PostgreSQL. No frontend, foram criados o provedor de autenticação e o *hook useAuth* (baseado em *onAuthStateChange*), além de um componente *ProtectedRoute* que protege as rotas privadas. As telas de login e cadastro passaram a usar *signInWithPassword/signUp*, e o logout passou a encerrar a sessão de fato. No backend, um *middleware* valida o JWT do Supabase nas rotas de áudio; os modelos de *logs* e *profiles* passaram a

operar sobre o PostgreSQL; e foram criados o *script* SQL com as tabelas, as políticas de **RLS** e o *trigger* de criação de perfil. A autenticação legada (bcrypt + SQLite) e suas dependências foram removidas.

Fase 2 — Estação de áudio multitrack (DAW) na web. Construiu-se uma página de mixagem (*/daw*) inspirada em softwares profissionais, com um **motor de áudio imperativo** (fora do ciclo de renderização do React) que gerencia o *AudioContext*, o barramento master e as faixas. Cada faixa possui sua própria cadeia de efeitos, ganho, *pan* e medidor. Os elementos de interface incluem: **forma de onda** com cursor de reprodução (picos calculados em **Web Worker** e desenhados em canvas em camadas), **medidores VU/pico** de entrada e saída, **analisador de espectro (FFT)**, **potenciômetros (knobs)** e **faders**. Foi definido um **catálogo de efeitos** servido pelo backend, com detecção automática dos filtros realmente disponíveis no *build* do FFmpeg; cada efeito é classificado conforme seu preview (exato em tempo real, aproximado ou apenas em renderização). O backend ganhou o *endpoint POST /audio/mix*, que serializa o projeto em JSON e monta um grafo *-filter_complex* — uma cadeia por faixa, somadas via *amix*, com limitador no master. Foram aplicadas as otimizações descritas na contextualização (ring buffer, laço único de animação com limitação adaptativa, *transferables*, ausência de atualização de estado do React durante a reprodução, desfazer/refazer em pilha e liberação de memória ao remover faixas).

Fase 3 — Conversão de áudio de maior qualidade. O *endpoint* de conversão passou a aceitar e validar, por codec, opções de **bitrate** (formatos com perdas), **sample rate** (44,1/48/96 kHz) e **profundidade de bits** (16/24/32, para WAV/FLAC), preservando o *resampler* de alta qualidade já existente. Esses controles foram expostos na interface das telas de Conversão e Pitch, exibindo somente as opções pertinentes ao formato escolhido.

Fase 4 — Avaliação de UX/UI e correções. Foi produzido um documento de auditoria (heurísticas de usabilidade, acessibilidade e gargalos de desempenho atuais e futuros) e implementadas as correções de maior impacto e menor custo: nomes acessíveis em botões somente com ícone, associação de rótulos aos campos dos formulários de autenticação, foco visível consistente, respeito à preferência de **redução de movimento**, atalho de teclado (barra de espaço para tocar/pausar) e **limite de tamanho de upload** no servidor.

Atividades de acabamento. Foram ainda implementados: o *script dev.js* para execução conjunta dos servidores; a correção e padronização do **favicon** e da identidade visual (logo ao lado do nome no cabeçalho); e o redesenho dos controles

— *knobs* com aparência de dial metálico e *faders* de equalização no estilo de mesas de mixagem — mantendo a paleta do projeto.

4 RESULTADOS E DISCUSSÕES

Os resultados foram verificados por critérios objetivos de testes e *build*.

Backend — testes automatizados. A suíte do backend executa **19 testes aprovados** (*node --test*), cobrindo unidade e integração. Destacam-se: a validação de *pitch*, formato e qualidade; a geração correta das *strings* de filtro e do grafo - *filter_complex* para 1, 2 e N faixas; a rejeição de projetos inválidos; e dois testes de integração que **executam o FFmpeg de verdade** — um renderiza uma mixagem de duas faixas com efeitos e outro confirma, por leitura dos metadados do arquivo, uma conversão em **48 kHz e WAV 24-bit**.

Frontend — build e análise estática. O *build* de produção do frontend conclui sem erros e a análise do ESLint permanece **limpa**, inclusive após a introdução da DAW, do motor de áudio, dos *workers* e dos novos componentes de interface.

Catálogo de efeitos. O *endpoint* de catálogo expõe **17 efeitos** efetivamente disponíveis no *build* do FFmpeg utilizado, organizados em Filtros, EQ, Dinâmica e Realce. A detecção automática de filtros revelou e tratou a **ausência** de dois filtros (*aexciter* e *asubboost*) nesse *build*, que foram corretamente omitidos do catálogo em vez de gerar comandos inválidos.

Discussão de trade-offs. Dois pontos técnicos merecem destaque. Primeiro, o filtro *amix* da versão de FFmpeg embutida não suporta a opção *normalize*; para preservar os ganhos por faixa na soma, aplicou-se uma compensação de volume proporcional ao número de faixas, com o limitador do master protegendo contra picos. Segundo, alguns efeitos (como *loudnorm*, *dynaunorm* e *firequalizer*) não possuem equivalente exato em tempo real no navegador; por isso, recebem o rótulo de “**apenas renderização**” na interface, deixando claro ao usuário que o resultado fiel virá do arquivo exportado, enquanto os filtros de EQ, ganho e dinâmica oferecem **preview ao vivo**.

Segurança e arquitetura. A camada de autenticação passou a operar com sessão JWT gerenciada e RLS no banco, mantendo a *service-role key* restrita ao servidor e expondo apenas a *anon key* no cliente. A reorganização em MVC tornou o código mais legível e preparado para crescer, com responsabilidades isoladas por camada.

5 CONSIDERAÇÕES FINAIS

O objetivo da etapa foi alcançado. O SoundCon evoluiu de um conversor de áudio para uma plataforma web com autenticação em nuvem, banco PostgreSQL com regras de acesso, conversão de maior qualidade e uma estação de mixagem multitrack que combina preview em tempo real (Web Audio) com renderização fiel (FFmpeg), tudo sobre um backend reorganizado em arquitetura MVC e com cobertura de testes.

Do ponto de vista formativo, a atividade exercitou de maneira integrada diversos conhecimentos do curso: engenharia de software incremental e arquitetura em camadas; segurança e autenticação de aplicações; processamento digital de sinais e a Web Audio API; e, sobretudo, a aplicação prática de **estruturas de dados** (buffers circulares, pilhas, *typed arrays*) e técnicas de otimização para que o serviço funcione bem mesmo em computadores modestos — premissa central do caráter extensionista do projeto, que busca acessibilidade de uso.

Como continuidade, recomenda-se: a virtualização das faixas na DAW para cenários com muitas trilhas; a renderização da mixagem de forma assíncrona, com progresso e cancelamento; a divisão do *bundle* (carregamento sob demanda da DAW e do Supabase); a adoção de testes automatizados de acessibilidade no processo de integração contínua; e a ativação final do ambiente Supabase em produção, com a configuração das chaves e a execução do *script* de banco.

6 REFERÊNCIAS

FFMPEG. **FFmpeg Filters Documentation**. Disponível em:

<https://ffmpeg.org/ffmpeg-filters.html>. Acesso em: 9 jun. 2026.

FFMPEG. **FFmpeg Resampler Documentation**. Disponível em:

<https://ffmpeg.org/ffmpeg-resampler.html>. Acesso em: 9 jun. 2026.

SUPABASE. **Supabase Documentation: Auth and Row Level Security**. Disponível em: <https://supabase.com/docs>. Acesso em: 9 jun. 2026.

MOZILLA. **Web Audio API — MDN Web Docs**. Disponível em:

https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API. Acesso em: 9 jun. 2026.

META (REACT). **React Documentation**. Disponível em: <https://react.dev>. Acesso em: 9 jun. 2026.

VITE. **Vite Documentation**. Disponível em: <https://vitejs.dev>. Acesso em: 9 jun. 2026.

INSTITUTO FEDERAL DO PARANÁ. **Normas para apresentação de trabalhos acadêmicos do IFPR**. Disponível em: <http://reitoria.ifpr.edu.br/wp->

content/uploads/2010/05/normas_ifpr_completa_alta_impressao.pdf. Acesso em: 9 jun. 2026.